# Selfie Pipes: Phased Implementation Plan

Nov 2, 2026 · @Creeplyn

Seven phases (0–6) take the course base code ( `selfie_02_with_fork.c` , saved as `selfie.c` ), which already has `fork()` and `wait()` , to blocking pipes between processes. Each phase ends at a gate: `selfie.c` builds with gcc, compiles itself with zero warnings, still runs itself (selfie-on-selfie), and passes its own tests plus every earlier phase's tests. Every phase and all 15 tests below were built and run on a copy of that base code, and the expected outputs are the ones those runs produced. Line numbers refer to the unmodified base file.

## What changed for this base code

The pipe design is the same; the groundwork under it changed. This base already has process states (fields 32–37, `STATUS_*` ), a `blocked_contexts` list with `block_context()` / `unblock_context()` , `fork()` (syscall 215) and `wait()` (216). Its scheduler switches process after every syscall. Four bugs in that code stop pipes from working, and the plan now fixes them first.

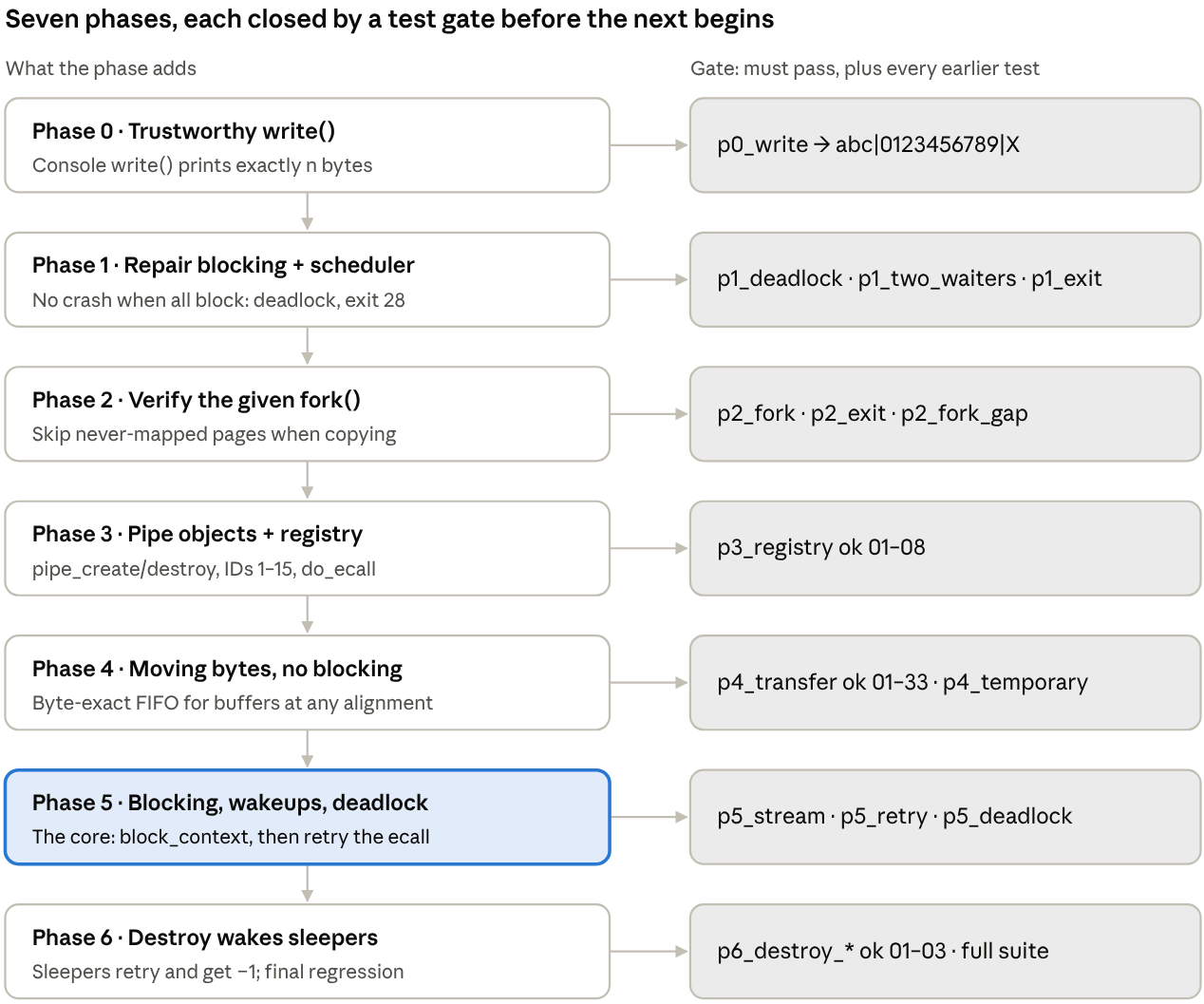
| Bug | Where (base file) | Effect | Fixed in |
|---|---|---|---|
| Scheduler picks a null or blocked context | `mipster()` , line 11880 | Selfie crashes when every remaining process is blocked; without the fix, pipe tests hang or crash | Phase 1 |
| Blocked-list head not updated | `block_context` / `unblock_context` , lines 11210, 11236 | List corruption once two processes are blocked | Phase 1 |
| Exit status written to an unmapped page | `implement_exit` , line 7604 | Crash when the waiting parent's status variable was never touched | Phase 1 |

| Bug | Where (base file) | Effect | Fixed in |
|---|---|---|---|
| `fork()` copies pages that were never mapped | `implement_fork`, lines 8085, 8093 | Crash after a large, partly used `malloc` | Phase 2 |

Other changes: `do_ecall` (line 10122) must be told which argument registers each pipe syscall reads (Phase 3). Several memory helpers were renamed or removed, so guest memory is accessed with `load_virtual_memory` / `store_virtual_memory` (Phase 4). Pipe wait queues get their own context field 38, because `block_context` reuses the next/prev pointers (Phase 5).

Outside this plan: `wait()` treats a child exit code of 0 as "no child exited yet", so a child that exits with 0 before its parent calls `wait()` leaves the parent blocked. After Phase 1 selfie reports this as a deadlock instead of crashing; the real fix belongs in `wait()`.

# At a glance




implementation roadmap · 7 phases, 7 gates

Read it top to bottom: each phase uses what the one above it built, and the next phase starts only after its gate passes. Phase 5 is the core of the architecture; Phases 0–4 make it testable, and Phase 6 closes the last gap.

# How to test every phase

A phase is done only when all four gate steps pass and every earlier phase's tests still give their expected output.

```
# 1. builds
gcc -w -O2 -m64 -D'uint64_t=unsigned long' selfie.c -o selfie
# 2. compiles itself: the output must contain no "error" or "warning" line
./selfie -c selfie.c
# 3. still runs itself (selfie-on-selfie); hello.c just writes "hello from
selfie"
./selfie -c selfie.c -m 64 -c hello.c -m 1
# 4. the phase's tests (show only the program's own output)
./selfie -c tests/p3_registry.c -m 2 | sed -n '/>>>>/,/<<<</p'
```

Pass rules:

- Step 2 passes when no line contains "error" or "warning". Step 3 passes when it prints "hello from selfie" and both terminating lines say exit code 0.
- A test passes when its output matches the expected output exactly: every line is `ok NN`, no `FAIL`, nothing missing, in the order given. The `terminating … with exit code N` line must show the expected code.
- This base switches process after every syscall, so tests with several processes print their lines interleaved. The order is still fixed, and the expected orders below are the measured ones.
- Regression: re-run every earlier phase's tests at each gate. The one planned exception is `p4_temporary.c`, which is valid only in Phase 4 and must deadlock from Phase 5 on.
- Tests are C* programs (selfie compiles them), so they use no `for`, `&&`, `||`, arrays or structs.
- A syscall's −1 reaches the program as 4294967295, because selfie shrinks syscall results to 32 bits (`sign_shrink(-1, SYSCALL_BITWIDTH)`). The tests compare against `MINUS1`.
- A crash of selfie itself shows as "Segmentation fault" with shell exit status 139 and no `terminating` line. Check it with `echo $?` right after the command.

Shared helpers. They are already pasted into every test that needs them (all except `p0_write.c`, `p1_exit.c`, `p1_deadlock.c`, `p5_stream.c` and `p5_deadlock.c`). `check` sends each line in a single `write` call, so lines from different processes never mix letter by letter:

```
uint64_t* out;
uint64_t MINUS1 = 4294967295; // -1 as seen by programs: syscall results are
32-bit
```

```
void put(uint64_t c) {
  *out = c;
  write(1, out, 1);
}

void check(uint64_t id, uint64_t pass) {
  // the whole line goes out in ONE write, so lines from different processes
never mix
  if (pass) {
    *out = 'o' + 'k' * 256 + ' ' * 65536 + ('0' + id / 10) * 16777216
      + ('0' + id % 10) * 4294967296 + 10 * 1099511627776;
    write(1, out, 6);
  } else {
    *out = 'F' + 'A' * 256 + 'I' * 65536 + 'L' * 16777216 + ' ' * 4294967296
      + ('0' + id / 10) * 1099511627776 + ('0' + id % 10) * 281474976710656 +
10 * 72057594037927936;
    write(1, out, 8);
  }
}

void spin(uint64_t n) {
  // burn CPU so the timer interrupt hands the CPU to another process
  uint64_t i;
  i = 0;
  while (i < n)
    i = i + 1;
}
```

Every test that uses `check` sets `out = malloc(8);` first in `main`.

## Phase 0 — Trustworthy console output

**Outcome:** `write(1, buf, n)` prints exactly `n` bytes for every `n`.

**Goal:** every later test reports through `write()`, so fix the existing bug first. Without this fix a correct pipe looks broken: the prototype printed "byte byts" until it was fixed.

### Requirements

1. **R0.1** In `implement_write` (line 7814), after `copy_buffer` succeeds, terminate again exactly after `size` bytes: `store_character((char*) IO_buffer, size, 0);`. Reason: `copy_buffer` copies whole 8-byte words and overwrites the terminator written before it.

2. **R0.2** Nothing else changes: same return value, same −1 error path.
3. **R0.3** The gate passes (build, self-compile, selfie-on-selfie).

**Testing protocol**

1. Before changing anything, run `p0_write.c` on the unmodified `selfie.c`. Expected: `abcdefgh|0123456789|XY` (the bug, reproduced).
2. Apply R0.1, then run gate steps 1–3.
3. Run `p0_write.c` again. Expected: `abc|0123456789|X`, exit code 0.

```
// tests/p0_write.c
uint64_t main() {
  write(1, "abcdefghij", 3);
  write(1, "|", 1);
  write(1, "0123456789", 10);
  write(1, "|", 1);
  write(1, "XY", 1);
  write(1, "\n", 1);
  return 0;
}
```

Passes when: step 1 shows the bug, which proves the test can detect it; step 3 matches exactly (R0.1, R0.2); the gate passes (R0.3).

## Phase 1 — Repair blocking and scheduling in the base code

**Outcome:** the scheduler only ever runs contexts that are READY or RUNNING. Two processes can be blocked at the same time. When every remaining process is blocked, selfie reports a deadlock (exit code 28) instead of crashing.

**Goal:** pipes put processes to sleep through the base's `block_context()` / `unblock_context()`, so that code must be correct first. Measured: without these repairs, `p5_stream` and `p5_retry` hang and `p5_deadlock` crashes selfie.

**Requirements**

1. **R1.1** Keep what the base already has: fields 32–37, the `STATUS_*` constants, `get_status` / `set_status`, `blocked_contexts`, `N_BLOCKED_CONTEXTS` and `next_pid`. Don't add a second status field: the names would clash.

2. **R1.2** `block_context` (line 11210): set the context's prev pointer to 0 and always make it the new head of `blocked_contexts`. The base only sets the head when the list is empty.
3. **R1.3** `unblock_context` (line 11236): when the context was the head of the blocked list (its prev is 0), set `blocked_contexts` to its next. Set `used_contexts`' prev pointer only when `used_contexts` is not 0.
4. **R1.4** At the end of `init_context` (line ~11167), set parent process, child exit code and child PID to 0. Contexts are reused from `free_contexts`, and `smalloc` does not zero memory.
5. **R1.5** `implement_exit` (line 7604): write the exit status with `map_and_store` instead of `store_virtual_memory`, because the waiting parent's status variable may sit on a page it never touched.
6. **R1.6** Add `next_ready_context(from)`:
   a. If `from` is RUNNING, mark it READY.
   b. If `from` is READY and has a next context, return that next context.
   c. Otherwise return the head of `used_contexts` (0 when no context is ready). After `block_context` or `delete_context`, `from`'s next pointer belongs to another list, so it must not be followed.
7. **R1.7** In `mipster()` (line 11880), replace the `TODO: scheduler should go here` block:
   a. `to_context = next_ready_context(from_context)`. If it is 0, call `report_deadlock()`.
   b. In the `EXIT` branch, if `used_contexts` is 0 while `N_BLOCKED_CONTEXTS > 0`, call `report_deadlock()` too.
   c. `report_deadlock()` prints `deadlock: N context(s) blocked and none ready` and returns `EXITCODE_DEADLOCK = 28`, added after `EXITCODE_UNCAUGHTEXCEPTION` (line 2492).
8. **R1.8** Keep the base's policy of switching after every syscall.

### Testing protocol

1. Negative check on the unmodified base: `p1_deadlock.c` and `p1_two_waiters.c` both crash selfie (Segmentation fault, exit status 139, no program output).
2. Apply R1.2–R1.7 and run gate steps 1–3.
3. `p1_deadlock.c`: `before wait`, then `deadlock: 1 context(s) blocked and none ready`, exit code 28.
4. `p1_two_waiters.c`: `ok 01`, `ok 02`, `ok 03`, exit code 0.

5. `p1_exit.c`: `survived timer interrupts`, exit code 7.
6. Regression: `p0_write.c`.

```
// tests/p1_deadlock.c  (no helpers needed)
// wait() with no child: nobody can ever wake this process
uint64_t main() {
  uint64_t* st;

  st = malloc(8);
  write(1, "before wait\n", 12);
  wait(st);
  write(1, "after wait\n", 11);

  return 0;
}
```

```
// tests/p1_two_waiters.c  (helpers pasted above)
// two contexts blocked at the same time (parent and child both in wait)
uint64_t main() {
  uint64_t* st;

  out = malloc(8);
  st = malloc(8);

  if (fork() == 0) {
    if (fork() == 0) {
      spin(600000);
      check(1, 1);   // grandchild finishes first
      exit(3);
    }
    wait(st);
    check(2, 1);     // child wakes after the grandchild exits
    exit(4);
  }

  wait(st);
  check(3, 1);       // parent wakes last
  return 0;
}
```

```
// tests/p1_exit.c  (no helpers needed)
uint64_t main() {
  uint64_t i;
  i = 0;
  // long enough for many timer interrupts with a single context
  while (i < 2000000)
    i = i + 1;
  write(1, "survived timer interrupts\n", 26);
  return 7;
}
```

Passes when: step 1 crashes as described, which proves the tests catch the bugs; `p1_deadlock` reports the deadlock (R1.6, R1.7); `p1_two_waiters` prints all three lines, so two blocked processes and the exit status write work (R1.2–R1.5); `p1_exit` still ends with exit code 7 (R1.8); and the gate passes.

## Phase 2 — Verify and repair the given fork()

**Outcome:** the base's `fork()` gives the child a private copy of code, data, heap and stack, even when the parent has heap pages it reserved but never touched. The child gets 0; the parent gets the child's PID (1, 2, …).

**Goal:** `fork()` is already in the base (syscall 215, `implement_fork` at line 8057). Don't rewrite it: check that it does what pipes need and fix the one crash.

### Requirements

1. **R2.1** Keep the given `fork()`: `SYSCALL_FORK = 215`, `emit_fork`, `implement_fork`, and PIDs from `next_pid`. The boot process is PID 0, so the first child is PID 1.
2. **R2.2** In both copy loops (lines 8085 and 8093), copy a word only if `is_virtual_address_mapped(get_pt(context), vaddr)`. The loops walk every word between the lowest and highest page, and `load_virtual_memory` on a page that was never mapped crashes selfie.
3. **R2.3** Change nothing else in `fork()`.

### Testing protocol

1. Negative check on the Phase 1 build: `p2_fork_gap.c` crashes selfie (Segmentation fault, exit status 139).
2. Apply R2.2 and run gate steps 1–3.

3. `p2_fork.c`: `ok 01`, `ok 04`, `ok 02`, `ok 05`, `ok 03`, `ok 06`, `ok 07`, `ok 08`, exit code 0. Every syscall switches process, so child and parent alternate lines until the child exits; the parent prints 06–08 after its `spin`.
4. `p2_exit.c`: `ok 01`, `ok 02`, exit code 5. The parent exits first; the machine keeps running the child and reports the exit code of the context that exited last.
5. `p2_fork_gap.c`: `ok 01`, `ok 02`, exit code 0.
6. Regression: the Phase 0–1 tests.

```
// tests/p2_fork.c  (helpers pasted above)
uint64_t x;

uint64_t main() {
  uint64_t pid;
  uint64_t local;
  uint64_t* heap;

  out = malloc(8);
  heap = malloc(8);

  x = 1;
  local = 42;
  *heap = 7;

  pid = fork();

  if (pid == 0) {
    // child: sees copies of everything, then changes its copies
    check(1, x == 1);
    check(2, local == 42);
    check(3, *heap == 7);
    x = 2;
    local = 43;
    *heap = 9;
    exit(0);
  }

  check(4, pid > 0);
  check(5, pid != MINUS1);

  // let the child run (timer interrupt), then confirm our copies are
untouched
```

```
  spin(400000);

  check(6, x == 1);
  check(7, local == 42);
  check(8, *heap == 7);

  return 0;
}
```

```
// tests/p2_exit.c  (helpers pasted above)
uint64_t main() {
  out = malloc(8);

  if (fork() == 0) {
    // child outlives the parent
    spin(400000);
    check(2, 1);
    exit(5);
  }

  check(1, 1);
  return 0;
}
```

```
// tests/p2_fork_gap.c  (helpers pasted above)
// fork after a large malloc that was only partly touched
uint64_t main() {
  uint64_t* big;

  out = malloc(8);
  big = malloc(65536);
  *(big + 8000) = 7;   // touch only the end: untouched pages lie in between

  if (fork() == 0) {
    check(1, *(big + 8000) == 7);
    *(big + 8000) = 9;
    exit(0);
  }

  spin(600000);
  check(2, *(big + 8000) == 7);
  return 0;
}
```

Passes when:

- `p2_fork` `ok 01` – `ok 03`: the child saw copied data, stack and heap (R2.1).
- `p2_fork` `ok 04` – `ok 05`: the parent got a valid PID (R2.1).
- `p2_fork` `ok 06` – `ok 08`: the child's writes did not reach the parent's memory.
- `p2_exit` ends with exit code 5: the machine runs until the last process exits.
- The negative check crashes and `p2_fork_gap` then prints both lines: untouched pages are skipped (R2.2) and the copy is still private.
- The gate and the regression tests pass (R2.3).

## Phase 3 — Pipe objects and the registry

**Outcome:** `pipe_create()` returns the lowest free ID (1–15), and `pipe_destroy(id)` frees it. Both return −1 for an invalid ID or a full registry.

**Goal:** get the data structures and the ID lookup right on their own, before any byte moves through them.

**Requirements**

1. **R3.1** Add `SYSCALL_PIPE_CREATE = 500`, `SYSCALL_PIPE_DESTROY = 503` and their prototypes next to the other syscall IDs (before the `DIRFD_AT_FDCWD` comment, line ~1311); they don't clash with `fork` (215) or `wait` (216). Add `emit_pipe_create()` (no arguments) and `emit_pipe_destroy()` (one argument: `a0` = ID), and call both from the bootstrapping list after `emit_open()` (line ~6344).
2. **R3.2** The pipe object is 9 words from `zmalloc`: id, buffer, r, w, n, readers head/tail, writers head/tail. Document it with a layout comment like the machine context's, and add `get_` / `set_` accessors for id, buffer, r, w and n. The ring storage is `zmalloc(round_up(PIPE_CAPACITY, 8))`, with `PIPE_CAPACITY = 16` bytes so blocking is easy to trigger later.
3. **R3.3** The registry `pipe_table` is allocated with `zmalloc(MAX_PIPES * 8)` on the first create. `MAX_PIPES = 16`, and ID 0 is never valid.
4. **R3.4** `lookup_pipe(id)` returns 0 if the table doesn't exist yet, if `id == 0`, if `id >= MAX_PIPES` (which also rejects −1), or if the slot is empty.
5. **R3.5** `implement_pipe_create` takes the lowest free slot and returns its ID. When the registry is full it returns −1. Add `syscall_return(context, value)` (sets `a0`, advances the PC) for all pipe syscalls.
6. **R3.6** `implement_pipe_destroy` returns −1 for an invalid ID. Otherwise it clears the slot and returns 0. Waking sleepers comes in Phase 6. The storage is not freed because selfie has no free list, so leave a TODO.
7. **R3.7** Put the kernel code just before `handle_system_call` (line 11762), where `map_page`, `palloc`, `map_and_store` and `block_context` are already defined. Add both dispatch branches before the `SYSCALL_EXIT` branch.
8. **R3.8** New for this base: `do_ecall` (line 10122) checks the argument registers of every syscall and assumes `a0` – `a2` for any syscall it doesn't know. Declare the pipe syscalls: `pipe_create` reads no register and only writes `a0` (`write_register(REG_A0)`), and `pipe_destroy` reads only `a0`, like `exit` and `wait`. `pipe_write` and `pipe_read` read `a0` – `a2`, which is the default.

### Testing protocol

1. Run gate steps 1–3.
2. Run `p3_registry.c`. Expected: `ok 01` to `ok 08`, exit code 0. If R3.8 is missing, selfie stops at the first `pipe_create` with `reading from uninitialized register a1`, then `… in presence of existing exception`, and exits with code 26.
3. Regression: the Phase 0–2 tests.

```
// tests/p3_registry.c  (helpers pasted above)
uint64_t main() {
  uint64_t p1;
  uint64_t p2;
  uint64_t i;

  out = malloc(8);

  p1 = pipe_create();
  p2 = pipe_create();
  check(1, p1 == 1);
  check(2, p2 == 2);
  check(3, pipe_destroy(p1) == 0);
  check(4, pipe_destroy(p1) == MINUS1);
  check(5, pipe_create() == 1);
  check(6, pipe_destroy(0) == MINUS1);
  check(7, pipe_destroy(99) == MINUS1);

  // IDs 3 .. 15 are still free
  i = 0;
  while (i < 13) {
    pipe_create();
    i = i + 1;
  }
  check(8, pipe_create() == MINUS1);

  return 0;
}
```

Passes when:

- `ok 01` – `ok 02`: lowest free ID (R3.3, R3.5).
- `ok 03` – `ok 04`: destroy works, and a second destroy is rejected (R3.6).
- `ok 05`: a freed ID is reused (R3.5).
- `ok 06` – `ok 07`: out-of-range IDs are rejected (R3.4).
- `ok 08`: a full registry returns −1 (R3.5).
- The program runs at all: `do_ecall` accepts the new syscalls (R3.8).
- The gate passes (R3.1, R3.7) and the regression tests pass.

# Phase 4 — Moving bytes without blocking

**Outcome:** `pipe_write` and `pipe_read` move `min(requested, available)` bytes in FIFO order, byte-exact, from and to buffers at any alignment. For now, when nothing can move they return 0.

**Goal:** get the data path and memory safety right in a single process, where every failure is deterministic and easy to trace, before adding concurrency.

**Requirements**

1. **R4.1** Add `SYSCALL_PIPE_WRITE = 501` and `SYSCALL_PIPE_READ = 502`, with 3-argument wrappers (`a0` = ID, `a1` = buffer, `a2` = size) that follow `emit_read`. Add them to the bootstrapping list after `emit_pipe_destroy()` and to the dispatch. `do_ecall` already treats them correctly (R3.8).
2. **R4.2** `guest_word_address(context, vaddr)` returns the address of the user's word that holds a given byte, or 0 if that byte isn't accessible:
   a. Round `vaddr` down to its word and check it with `is_virtual_address_valid` and `is_data_stack_heap_address`.
   b. If the page has never been touched, map a fresh `palloc()` frame with `map_page(context, get_page_of_virtual_address(waddr), …)`, as `handle_page_fault` does. This base renamed `page_of_virtual_address` and removed `translate_virtual_to_physical`.
   c. Move each byte through a one-word kernel scratch buffer (`pipe_word`): load the user's word with `load_virtual_memory`, read or change one byte with `load_character` / `store_character`, and, for reads, store the word back with `store_virtual_memory`. The other seven bytes stay as they were.
   d. Do not use `copy_buffer` for pipe data. It requires 8-byte-aligned addresses and copies whole words, so a 5-byte read would overwrite 3 bytes past the buffer.
3. **R4.3** Move data one byte at a time. Write: `buffer[w] = byte; w = (w + 1) % C; n = n + 1`. Read is the mirror image. Stop when `size` bytes have moved or the pipe is full (write) or empty (read).
4. **R4.4** Return values:
   a. `k`, the number of bytes moved. Short transfers are allowed, and callers loop.
   b. 0 when `size == 0`.
   c. −1 for an invalid ID.
   d. −1 for a bad address before any byte moved; the pipe stays unchanged.

   e. `k` for a bad address after some bytes moved.
5. **R4.5** Temporary rule: when nothing can move (a read of an empty pipe, a write to a full one), return 0 right away. Phase 5 replaces exactly these two branches.
6. **R4.6** Check the ID and `size == 0` before checking for empty/full, so those calls can never block once Phase 5 exists.

### Testing protocol

1. Run gate steps 1–3.
2. Run `p4_transfer.c`. Expected: `ok 01` to `ok 33`, exit code 0.
3. Run `p4_temporary.c`. Expected: `ok 01` to `ok 03`, exit code 0.
4. Regression: the Phase 0–3 tests.

```
// tests/p4_transfer.c  (helpers pasted above)
// single process; never blocks, so it stays valid after Phase 5
uint64_t main() {
  uint64_t p;
  uint64_t* buf;
  uint64_t* big;
  uint64_t msg;

  out = malloc(8);
  buf = malloc(8);

  p = pipe_create();

  check(1, pipe_write(p, "ABCDEFGHIJ", 10) == 10);
  *buf = 0;
  check(2, pipe_read(p, buf, 4) == 4);
  check(3, *buf == 'A' + 'B' * 256 + 'C' * 65536 + 'D' * 16777216);
  *buf = 0;
  check(4, pipe_read(p, buf, 100) == 6);
  check(5, *buf == 'E' + 'F' * 256 + 'G' * 65536 + 'H' * 16777216 + 'I' *
4294967296 + 'J' * 1099511627776);

  // short write: only 16 bytes fit
  msg = (uint64_t) "abcdefghijklmnopqrstuvwxyz0123456789ABCD";
  check(6, pipe_write(p, (uint64_t*) msg, 40) == 16);
  check(7, pipe_read(p, buf, 8) == 8);
  check(8, *buf == *((uint64_t*) msg));
  check(9, pipe_read(p, buf, 8) == 8);
  check(10, *buf == *((uint64_t*) msg + 1));
```

```
// unaligned source
check(11, pipe_write(p, (uint64_t*) (msg + 3), 2) == 2);
*buf = 0;
check(12, pipe_read(p, buf, 2) == 2);
check(13, *buf == 'd' + 'e' * 256);

// a 1-byte read must not touch the other bytes of the word
check(14, pipe_write(p, "Q", 1) == 1);
*buf = 'x' + 'y' * 256 + 'z' * 65536;
check(15, pipe_read(p, buf, 1) == 1);
check(16, *buf == 'Q' + 'y' * 256 + 'z' * 65536);

// unaligned destination
check(17, pipe_write(p, "RS", 2) == 2);
*buf = 0;
check(18, pipe_read(p, (uint64_t*) ((uint64_t) buf + 5), 2) == 2);
check(19, *buf == 'R' * 1099511627776 + 'S' * 281474976710656);

// destination on a heap page the program never touched
big = malloc(16384);
check(20, pipe_write(p, "Z", 1) == 1);
check(21, pipe_read(p, big + 1024, 1) == 1);
check(22, *(big + 1024) == 'Z');

// bad address: error, and the byte stays in the pipe
check(23, pipe_write(p, (uint64_t*) 8, 1) == MINUS1);
check(24, pipe_write(p, "W", 1) == 1);
check(25, pipe_read(p, (uint64_t*) 8, 1) == MINUS1);
*buf = 0;
check(26, pipe_read(p, buf, 1) == 1);
check(27, *buf == 'W');

// zero sizes and bad IDs
check(28, pipe_write(p, buf, 0) == 0);
check(29, pipe_read(p, buf, 0) == 0);
check(30, pipe_write(99, buf, 1) == MINUS1);
check(31, pipe_read(99, buf, 1) == MINUS1);

check(32, pipe_destroy(p) == 0);
```

```
  check(33, pipe_write(p, buf, 1) == MINUS1);

  return 0;
}
```

```
// tests/p4_temporary.c  (helpers pasted above)
// Phase 4 only: empty/full return 0 instead of blocking
uint64_t main() {
  uint64_t p;
  uint64_t* buf;

  out = malloc(8);
  buf = malloc(8);
  p = pipe_create();

  check(1, pipe_read(p, buf, 1) == 0);
  check(2, pipe_write(p, "abcdefghijklmnopqrstuvwxyz", 26) == 16);
  check(3, pipe_write(p, "a", 1) == 0);

  return 0;
}
```

Passes when:

| Checks | What they prove | Requirements |
|---|---|---|
| `ok 01` – `ok 05` | FIFO order, exact bytes, short read | R4.3, R4.4 |
| `ok 06` – `ok 10` | a short write stops at the 16-byte capacity | R4.3, R4.4 |
| `ok 11` – `ok 13` | unaligned source buffer | R4.2 |
| `ok 14` – `ok 16` | a read leaves the rest of the word alone | R4.2, R4.3 |
| `ok 17` – `ok 19` | unaligned destination buffer | R4.2 |
| `ok 20` – `ok 22` | a never-touched heap page is mapped on demand | R4.2 |
| `ok 23` – `ok 27` | bad address → −1, no byte lost | R4.4 |
| `ok 28` – `ok 31` | size 0 and bad IDs | R4.4, R4.6 |
| `ok 32` – `ok 33` | a destroyed ID can't be used | R3.6 |

| Checks | What they prove | Requirements |
|---|---|---|
| `p4_temporary` `ok 01` – `ok 03` | empty read and full write return 0 for now | R4.5 |

The gate (R4.1) and the regression tests must pass too.

## Phase 5 — Blocking, wakeups and deadlock detection

**Outcome:** a read of an empty pipe or a write to a full one puts the process to sleep instead of returning 0. The next transfer in the other direction wakes it, and it retries. If every remaining process is asleep, selfie reports a deadlock (exit code 28) instead of hanging.

**Goal:** this is the core of the pipe architecture: a producer and a consumer that wait for each other without spinning.

**Requirements**

1. **R5.1** Add context field 38, next waiter: raise `CONTEXTENTRIES` from 38 to 39 (line 2233), add the row to the layout comment, add `get_next_waiter` / `set_next_waiter`, and set it to 0 at the end of `init_context`. The pipe queues need their own link because `block_context` reuses the next/prev pointers for the `blocked_contexts` list.
2. **R5.2** Queue offsets `PIPE_READERS = 5` and `PIPE_WRITERS = 7`: the head is at `pipe + q` and the tail at `pipe + q + 1`. `enqueue_waiter` appends the context in FIFO order through field 38, then calls `block_context(context)`, which moves it off `used_contexts`, marks it BLOCKED and counts it in `N_BLOCKED_CONTEXTS`.
3. **R5.3** `wake_all_waiters(pipe, q)` clears each queued context's field 38 and calls `unblock_context(context)`, which puts it back on `used_contexts` as READY. Then it empties the queue. Waking all of them is safe because each one rechecks.
4. **R5.4** Replace the two R4.5 branches: enqueue the context and return without advancing the PC and without touching `a0`. When the context runs again, its `ecall` executes again with `a0` – `a2` and `a7` unchanged, so the recheck comes for free.
5. **R5.5** After a write that moved at least one byte, wake the readers. After a read that moved at least one byte, wake the writers.
6. **R5.6** No lock is needed: the `implement_*` handlers run to completion inside the emulator, and the timer only fires between guest instructions. Checking and enqueuing is therefore already one atomic step. Say this in your report.
7. **R5.7** Deadlock reporting already exists from Phase 1 (R1.7); nothing to add here.

8. **R5.8** Add a `debug_pipes` flag (default 0) that prints `pid N blocks on pipe P` and `pipe P wakes pid N`. The protocol uses it in step 4.

### Testing protocol

1. Run gate steps 1–3.
2. Run `p5_stream.c`, which sends 58 bytes through the 16-byte pipe, so both sides block. Expected: `Pipes carry bytes from one process to another, in order.` and exit code 0.
3. Run `p5_retry.c`. Expected: `r2=A` then `r1=B`, exit code 0. The rule behind it: each reader prints exactly once, one gets A and the other gets B.
4. Set `debug_pipes = 1`, rebuild, and run `p5_retry.c` again. Expected trace: both readers block, both wake, the reader that lost the race blocks again, `r2=A`, it wakes again, `r1=B`.
5. Run `p5_deadlock.c`. Expected: `deadlock: 1 context(s) blocked and none ready`, exit code 28.
6. Planned change: `p4_temporary.c` now prints nothing and ends with the same deadlock message and exit code 28. That proves the R4.5 branches are gone.
7. Regression: `p0_write`, `p1_exit`, `p1_deadlock`, `p1_two_waiters`, `p2_fork`, `p2_exit`, `p2_fork_gap`, `p3_registry`, `p4_transfer`.

```
// tests/p5_stream.c  (no helpers needed)
uint64_t main() {
  uint64_t p;
  uint64_t sent;
  uint64_t got;
  uint64_t r;
  uint64_t msg;
  uint64_t len;
  uint64_t* buf;

  msg = (uint64_t) "Pipes carry bytes from one process to another, in
order.\n";
  len = 58;

  p = pipe_create();

  if (fork() == 0) {
    // child = writer: short writes are allowed, so loop until everything is
sent
```

```
    sent = 0;
    while (sent < len) {
      r = pipe_write(p, (uint64_t*) (msg + sent), len - sent);
      sent = sent + r;
    }
    exit(0);
  }

  // parent = reader, 5 bytes per call
  buf = malloc(8);
  got = 0;
  while (got < len) {
    r = pipe_read(p, buf, 5);
    write(1, buf, r);
    got = got + r;
  }

  pipe_destroy(p);

  return 0;
}
```

```
// tests/p5_retry.c  (helpers pasted above)
// two readers sleep on one pipe; each byte wakes both, only one gets it, the
other sleeps again
uint64_t p;

void reader(uint64_t id) {
  uint64_t* buf;

  buf = malloc(8);
  *buf = 0;

  pipe_read(p, buf, 1);

  // "rN=X" plus newline in one write
  *out = 'r' + ('0' + id) * 256 + '=' * 65536 + *buf * 16777216 + 10 *
4294967296;
  write(1, out, 5);

  exit(0);
}
```

```
uint64_t main() {
  out = malloc(8);

  p = pipe_create();

  if (fork() == 0)
    reader(1);
  if (fork() == 0)
    reader(2);

  spin(400000);
  pipe_write(p, "A", 1);
  spin(400000);
  pipe_write(p, "B", 1);
  spin(400000);

  return 0;
}
```

```
// tests/p5_deadlock.c  (no helpers needed)
// a reader with no writer: the scheduler must report a deadlock
uint64_t main() {
  uint64_t p;
  uint64_t* buf;

  p = pipe_create();
  buf = malloc(8);
  pipe_read(p, buf, 1);

  return 0;
}
```

Expected `debug_pipes` trace for step 4:

```
./selfie: pid 1 blocks on pipe 1
./selfie: pid 2 blocks on pipe 1
./selfie: pipe 1 wakes pid 1
./selfie: pipe 1 wakes pid 2
./selfie: pid 1 blocks on pipe 1
r2=A
./selfie: pipe 1 wakes pid 1
r1=B
```

Passes when:

- `p5_stream` prints the message exactly: blocking in both directions plus retries (R5.1–R5.5).
- In the trace, the reader that loses the race goes back to sleep: wake-all followed by a recheck (R5.3, R5.4).
- `p5_deadlock` and `p4_temporary` end with exit code 28 (R5.7, through R1.7).
- The gate and the regression tests pass.

One side effect to mention in your report: each retried `ecall` counts again in the context's syscall counter (`ec_syscall`).

## Phase 6 — Destroy wakes sleepers, and the final regression

**Outcome:** destroying a pipe wakes every process sleeping on it. Each one retries, finds the ID invalid and gets −1. The full test suite passes, including pipes running inside selfie-on-selfie.

**Goal:** a destroyed pipe must not leave anyone asleep forever. Then confirm the whole architecture end to end.

### Requirements

1. **R6.1** In `implement_pipe_destroy`, clear the registry slot first, then call `wake_all_waiters` for the readers and the writers.
2. **R6.2** Add no special case to read or write. The retried `ecall` reaches the invalid-ID path from R4.4 and returns −1.
3. **R6.3** Every test from Phases 0–5 still gives its expected output, and pipes work when selfie runs itself.

### Testing protocol

1. Negative check, before applying R6.1, on the Phase 5 build:
   a. `p6_destroy_reader.c` must print `ok 01`, `ok 03`, then the deadlock message, exit code 28.
   b. `p6_destroy_writer.c` must print `ok 01`, `ok 02`, then the deadlock message, exit code 28.
   c. This proves both tests catch a missing wakeup.
2. Apply R6.1 and run gate steps 1–3.

3. `p6_destroy_reader.c`: `ok 01`, `ok 02`, `ok 03`, exit code 0. `p6_destroy_writer.c`: `ok 01`, `ok 03`, `ok 02`, exit code 0 (the woken child prints 03 before the parent's 02, because every syscall switches process).
4. Full regression: every test from Phases 0–5. `p4_temporary.c` still deadlocks, as planned.
5. Pipes inside selfie-on-selfie:
   a. `./selfie -c selfie.c -m 64 -c tests/p5_stream.c -m 2` prints the message, and both terminating lines say exit code 0.
   b. The same command with `tests/p4_transfer.c` prints 33 `ok` lines.

```
// tests/p6_destroy_reader.c  (helpers pasted above)
// a reader blocked on an empty pipe gets -1 when the pipe is destroyed
uint64_t main() {
  uint64_t p;
  uint64_t* buf;

  out = malloc(8);
  buf = malloc(8);
  p = pipe_create();

  if (fork() == 0) {
    check(2, pipe_read(p, buf, 4) == MINUS1);
    exit(0);
  }

  spin(400000);
  check(1, pipe_destroy(p) == 0);
  spin(400000);
  check(3, pipe_destroy(p) == MINUS1);

  return 0;
}
```

```
// tests/p6_destroy_writer.c  (helpers pasted above)
// a writer blocked on a full pipe gets -1 when the pipe is destroyed
uint64_t main() {
  uint64_t p;

  out = malloc(8);
  p = pipe_create();
```

```
  if (fork() == 0) {
    check(1, pipe_write(p, "abcdefghijklmnopqrstuvwxyz", 26) == 16);
    check(3, pipe_write(p, "a", 1) == MINUS1);
    exit(0);
  }

  spin(400000);
  check(2, pipe_destroy(p) == 0);
  spin(400000);

  return 0;
}
```

Passes when: the negative check fails as described; both destroy tests print `ok 01` to `ok 03` (R6.1, R6.2); and steps 4–5 all match (R6.3).

Final sign-off:

- [ ] Gate steps 1–3 pass on the final `selfie.c`
- [ ] `p0_write` → `abc|0123456789|X`
- [ ] `p1_exit` → message, exit code 7; `p1_deadlock` → deadlock, exit code 28; `p1_two_waiters` → `ok 01` – `ok 03`
- [ ] `p2_fork` → `ok 01 04 02 05 03 06 07 08`; `p2_exit` → exit code 5; `p2_fork_gap` → `ok 01`, `ok 02`
- [ ] `p3_registry` → `ok 01` – `ok 08`
- [ ] `p4_transfer` → `ok 01` – `ok 33`; `p4_temporary` → deadlock, exit code 28
- [ ] `p5_stream` → the message; `p5_retry` → `r2=A`, `r1=B`; `p5_deadlock` → exit code 28
- [ ] `p6_destroy_reader` → `ok 01 02 03`; `p6_destroy_writer` → `ok 01 03 02`
- [ ] Pipes inside selfie-on-selfie (step 5)